RoCam

High Performance Vision-Guided Rocket Tracker

Zifan Si
Full Stack & DevOps

Jianqing Liu
Arch. Backend & Firmware

Mike Chen
Data Tooling & i18n

Xiaotian Lou
CV, ML & System Tests

Supervisor: Dr. Shahin Sirouspour • Instructor: Dr. Spencer Smith • TA: Tanya Djavaherpour

TEAM #3 SPACEY | MCMASTER UNIVERSITY SFWRENG 4G06 | MCMASTER ROCKETRY TEAM


USST
BOREALIS
BESSIE
QRET
DEVELON
CANADA'S FUTURE IN SPACE
CAT

THE CHALLENGE

Rockets reach Mach 3 in seconds.

Manual optical tracking is physically impossible for small-profile hobbyist rocketry. Commercial rigs cost upwards of $10,000.

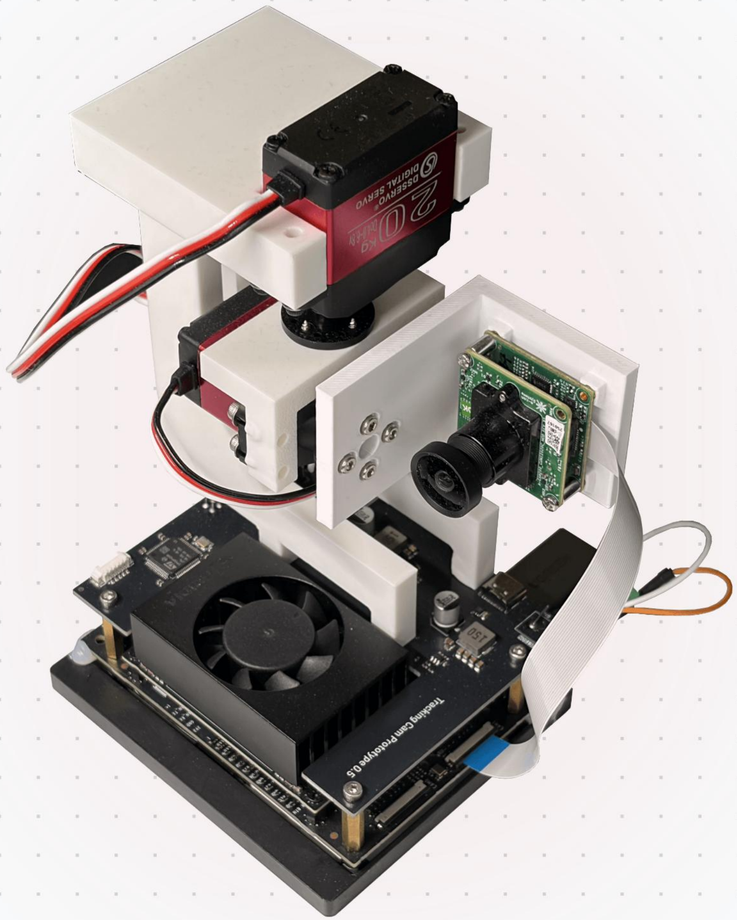
100 km
MAX ALTITUDE

>10 g
ACCELERATION

PRESENTATION ROADMAP

- 1. Problem & Solution
- 2. Architecture & Live Demo
- 3. Data, CV & ML Pipeline
- 4. Hardware & UI Control
- 5. Testing, Challenges & Future

THE ROCAM SOLUTION



Autonomous.
Accessible.
Accurate.

A fully autonomous, closed-loop optical tracking system engineered to meet strict performance constraints.

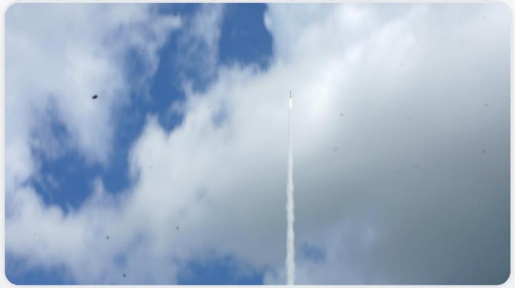
Autonomous Edge AI Tracking
Works in remote, offline, and hazardous conditions

Accurate and Precise
60Hz closed-loop control and digital stabilization

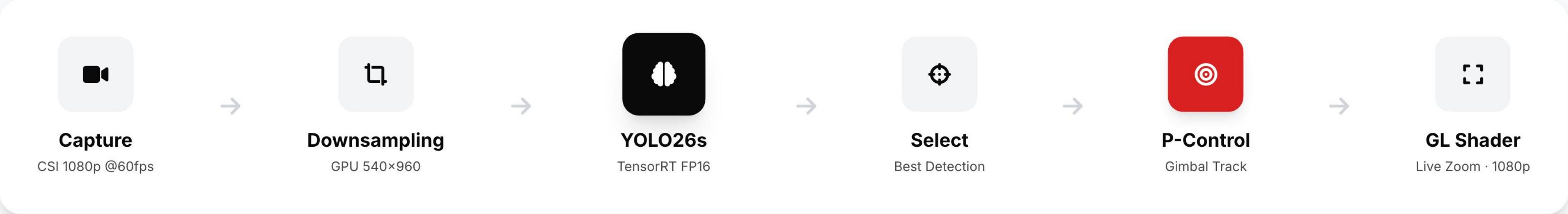
Low Cost
Total BOM under $500 CAD (vs $10k+ commercial)

LIVE DEMO

Optimized for extreme distance and strict latency budgets.



Raw frame — can you spot the rocket?



Why YOLO26s?

One-to-one detection head eliminates NMS entirely. 9.47M parameters engineered for FP16—only 9% accuracy loss on Jetson vs. 30% for alternative architectures.

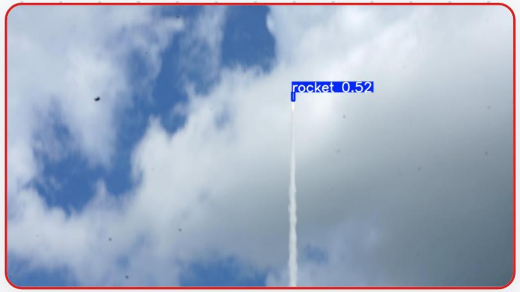
Full GPU Pipeline

DeepStream runs decode, downsampling, and TensorRT inference entirely on GPU. Confidence threshold filters noise; probe selects single best detection per frame.

Instant Visual Lock

GL fragment shader crops and zooms on target within one display frame. P-control drives gimbal via UART. Zero perceived tracking latency.

Optimized for extreme distance and strict latency budgets.



Target <0.1% of frame — detected at conf 0.52

Capture
CSI 1080p @60fps

→

Downsampling
GPU 540×960

→

YOLO26s
TensorRT FP16

→

Select
Best Detection

→

P-Control
Gimbal Track

→

GL Shader
Live Zoom · 1080p

Why YOLO26s?

One-to-one detection head eliminates NMS entirely. 9.47M parameters engineered for FP16—only 9% accuracy loss on Jetson vs. 30% for alternative architectures.

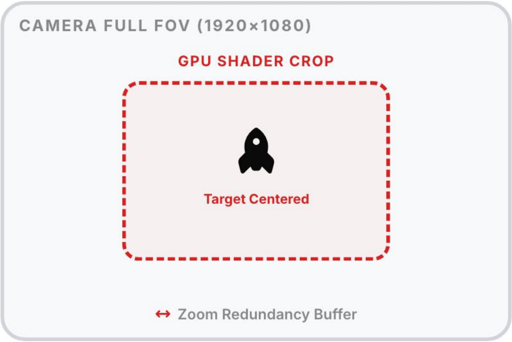
Full GPU Pipeline

DeepStream runs decode, downsampling, and TensorRT inference entirely on GPU. Confidence threshold filters noise; probe selects single best detection per frame.

Instant Visual Lock

GL fragment shader crops and zooms on target within one display frame. P-control drives gimbal via UART. Zero perceived tracking latency.

Optical redundancy enables perceptually instant tracking.



FOV Redundancy Model

Wide optical view guarantees target stays in frame while gimbal mechanically catches up

Detection Result

BBox (cx, cy, w, h) from YOLO @ 60fps

~1MS

GPU Shader Path

1. Compute tx, ty, scale from bbox
2. GL shader crops & centers frame
3. Livestream output → operator sees instant lock

~35MS

Gimbal Servo Path

1. P-control → delta pan/tilt
2. UART → STM32 → PWM servo
3. Mechanical motion re-centers optical FOV

Zero Perceived Latency

Operator sees target locked within one display frame (~16ms). Gimbal catches up transparently.

Custom labelling platform for rocket data.

All training data flows through our in-house **RoCam Labeler** — a full-stack annotation tool built from scratch to handle our unique dataset requirements.

SAM-Assisted Annotation

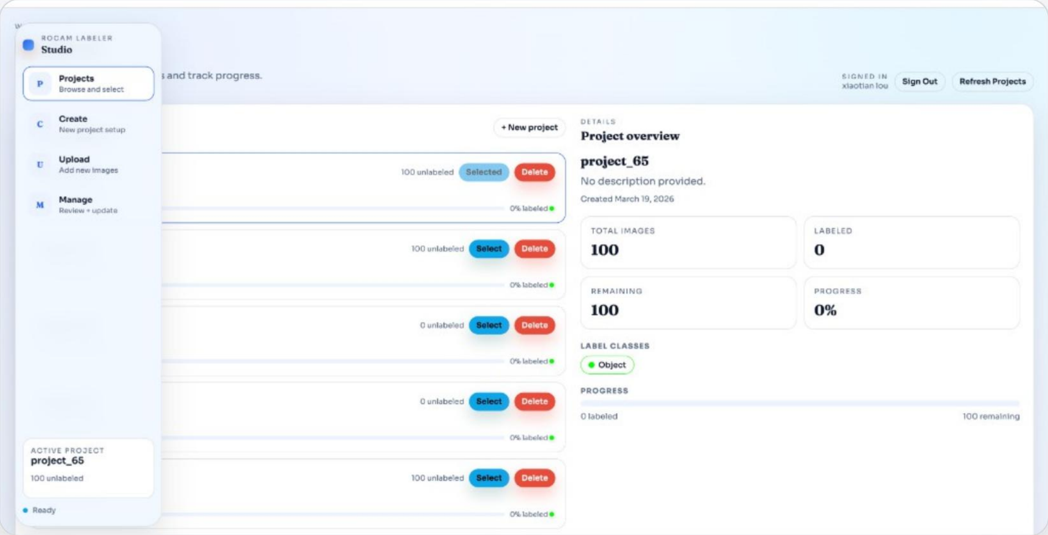
One-click segmentation masks via Meta SAM

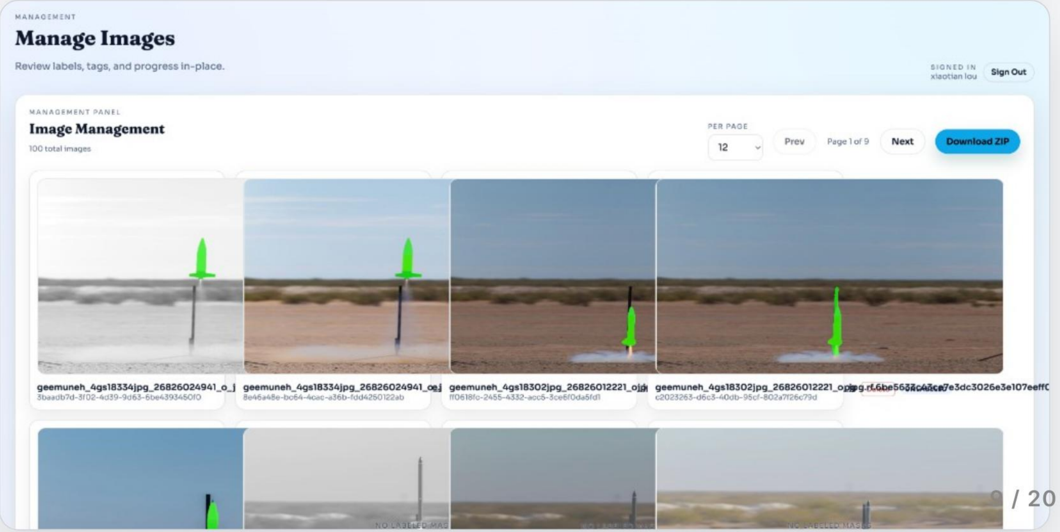
Project Management

Multi-project workspace with progress tracking

Batch Upload / Export

ZIP-based bulk import and YOLO-format export





96.2%

PEAK TRAINING ACCURACY

mAP@50 across 10+ experiments on H100

9% Deployment Gap

FP16 quantization on Jetson Orin Nano. Our best-in-training architecture lost **30%** at deployment—the production recipe held.

Aggressive Augmentation Training

YOLO26s trained end-to-end for **420 epochs** on H100 GPU. Data labelled via our custom **Labelling App** with SAM-assisted annotation.

Full Mosaic = 1.0 (Key Finding)

Every image trained as 2×2 mosaic, quadrupling effective object density. Disabled last 80 epochs for full-resolution refinement. All reduced-mosaic variants failed.

15,832 Images + 4,000 COCO Hard Negatives

25% of training set is pure background from COCO, forcing a harder decision boundary to suppress false positives in real-world deployment.

Multi-Scale + 7 Albumentations

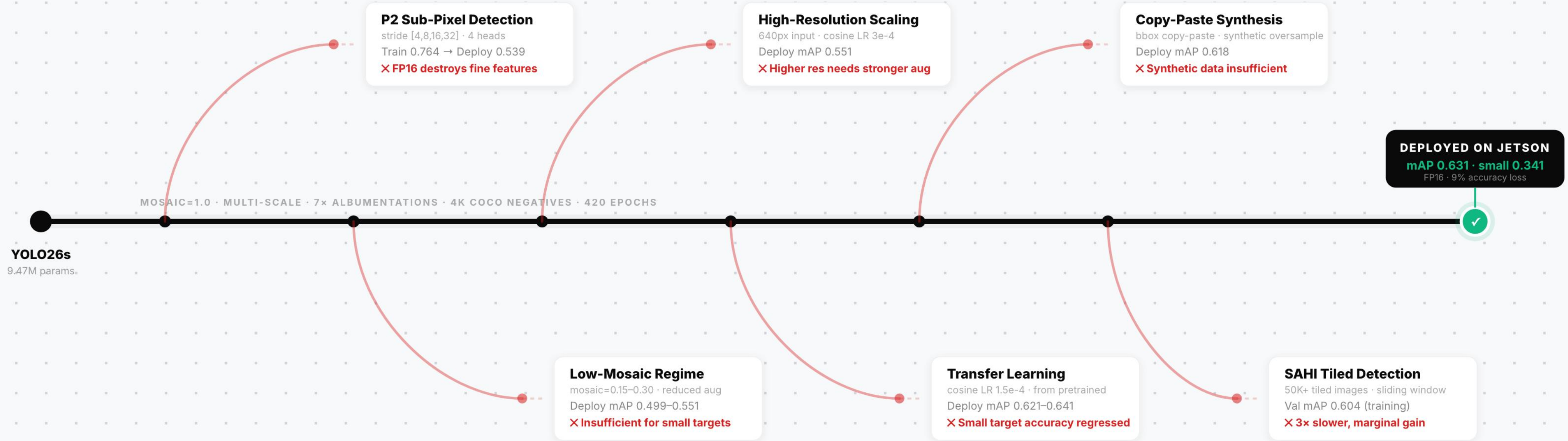
Variable resolution (544–960px), 180° rotation, motion blur, compression artefacts, and sensor noise transforms for real-world robustness.

94.3%	89.8%	60	9.47M	20.5	<1ms
PRECISION	RECALL	FPS PIPELINE	PARAMETERS	GFLOPS	SHADER LATENCY

MODEL DEVELOPMENT JOURNEY

Systematic Exploration. One Survivor.

Every promising architecture and training strategy tested against real edge deployment constraints

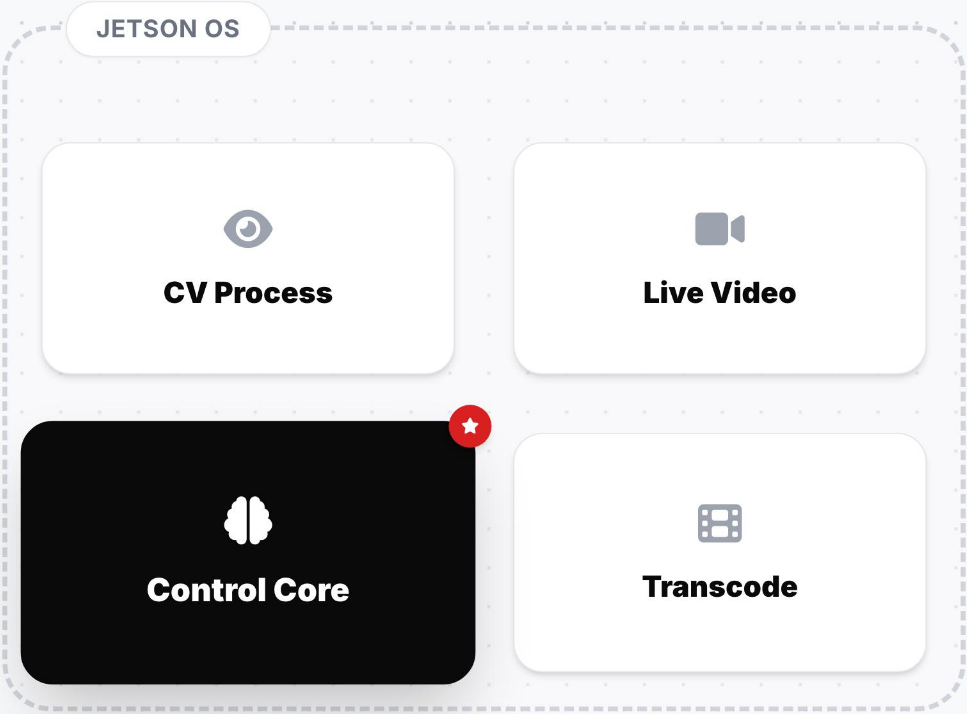


10+ EXPERIMENTS CONDUCTED

100+ GPU-HOURS ON H100

9% FP16 ACCURACY LOSS (BEST)

1 SURVIVED DEPLOYMENT



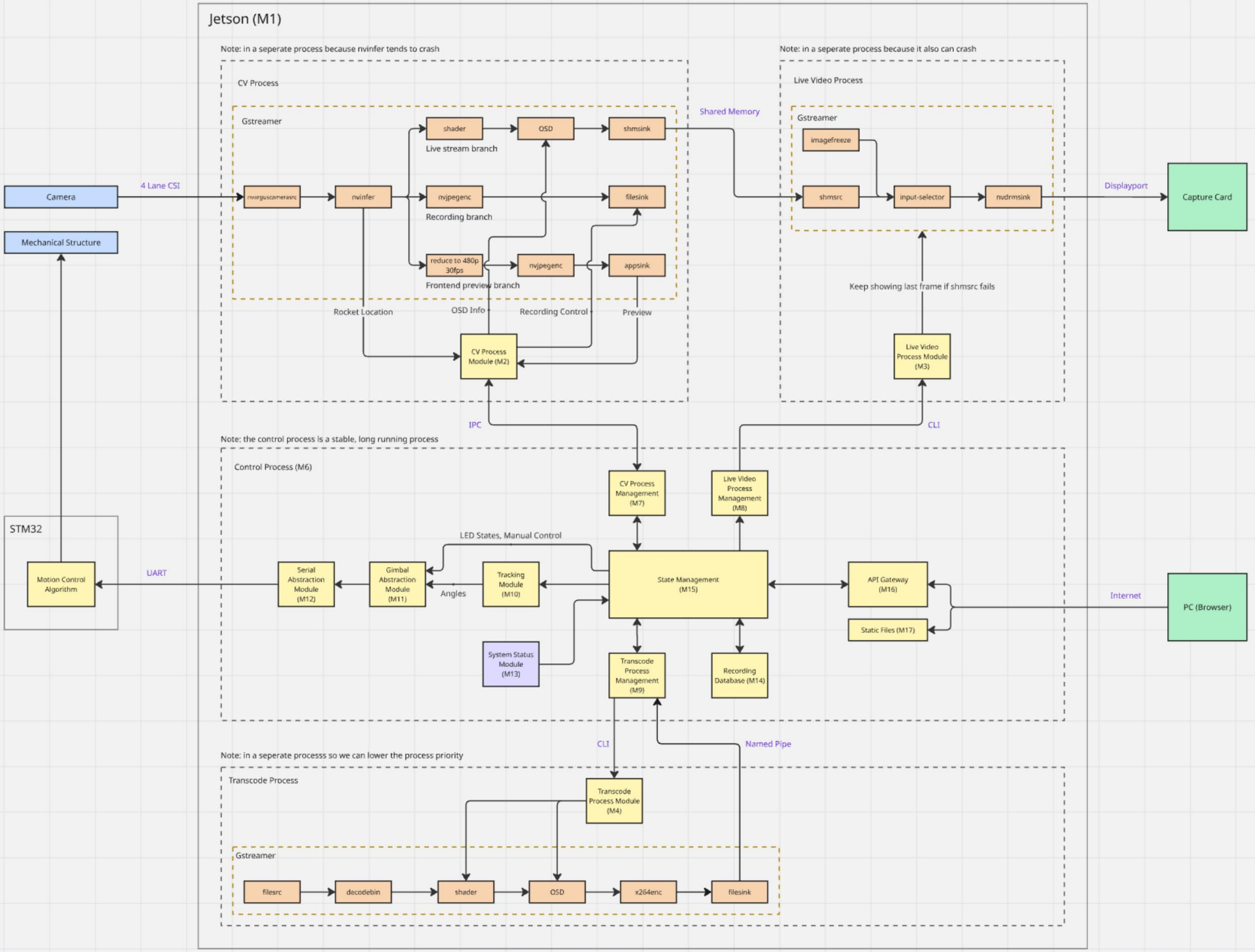
Built for isolation and real-time survival.

Independent Restart

If the CV inference crashes, the control process maintains connection to the gimbal, preventing lock-up.

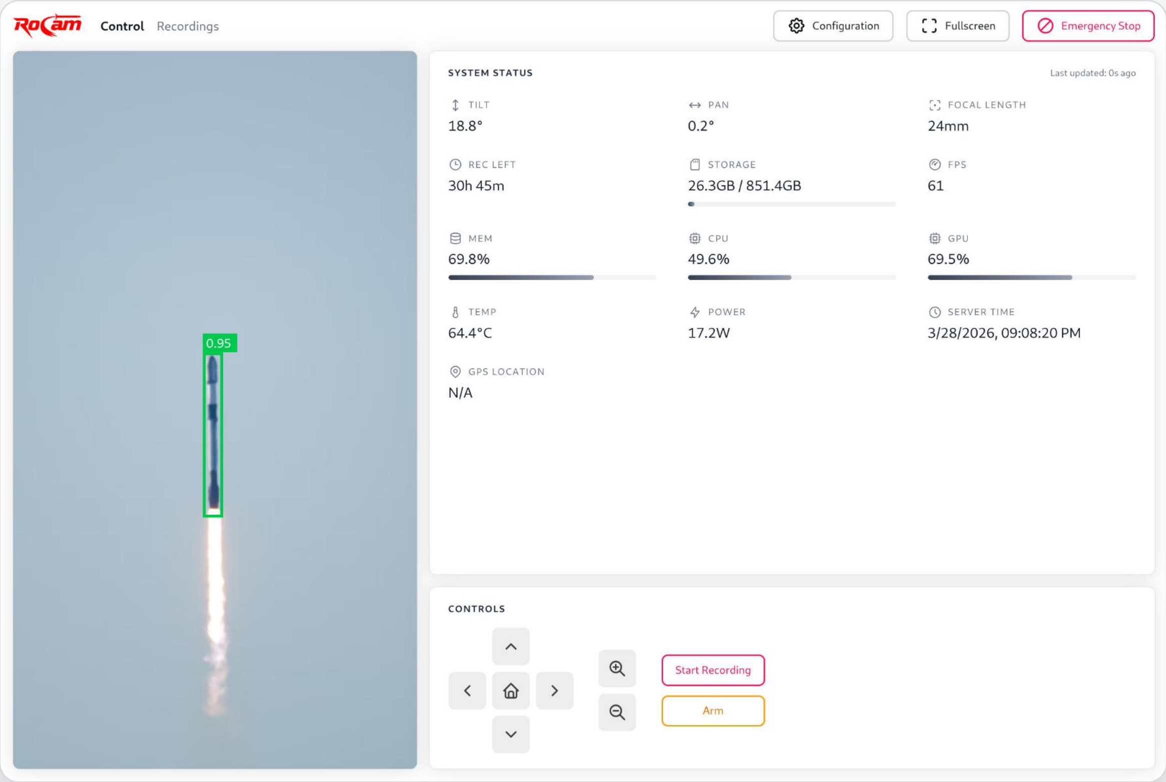
Zero-copy IPC

Shared-memory architecture allows massive 1080p frames to move between processes instantly.



Jetson (M1)
Note: in a seperate process because nvinfer tends to crash
CV Process
Gstreamer
shader
OSD
shmsink
Live stream branch
nvarguscamerasrc
nvinfer
nvjpegenc
filesink
Recording branch
reduce to 480p 30fps
nvjpegenc
appsink
Frontend preview branch
Rocket Location
OSD Info
Recording Control
Preview
CV Process Module (M2)
Shared Memory
Note: in a seperate process because it also can crash
Live Video Process
Gstreamer
imagefreeze
shmsrc
input-selector
nvdrmsink
Keep showing last frame if shmsrc fails
Live Video Process Module (M3)
Displayport
Capture Card
Camera
4 Lane CSI
Mechanical Structure
IPC
CLI
Note: the control process is a stable, long running process
Control Process (M6)
CV Process Management (M7)
Live Video Process Management (M8)
STM32
Motion Control Algorithm
UART
LED States, Manual Control
Serial Abstraction Module (M12)
Gimbal Abstraction Module (M11)
Angles
Tracking Module (M10)
State Management (M15)
API Gateway (M16)
Static Files (M17)
Internet
PC (Browser)
System Status Module (M13)
Transcode Process Management (M9)
Recording Database (M14)
CLI
Named Pipe
Note: in a seperate processs so we can lower the process priority
Transcode Process
Transcode Process Module (M4)
Gstreamer
filesrc
decodebin
shader
OSD
x264enc
filesink

MISSION CONTROL SOFTWARE



FIELD USABILITY FIRST

Command center in a browser.

REACT • TYPESCRIPT • VITE

Everything the operator touches lives in **Mission Control**. Before launch, the UI must let the operator confirm framing, arm the system, override the gimbal if needed, and understand system health immediately.

OPERATOR WORKFLOW

- Confirm framing
- Arm the tracker
- Override motion
- Read health instantly

Low-latency live preview

See framing and target lock with minimal delay during setup and tracking.

Direct manual override

Take control of the gimbal immediately when the operator needs to reframe.

Real-time health telemetry

Arming, device, network, and storage state stay visible at a glance.

Bilingual EN / FR

The same control surface supports English and French operators in the field.

RECORDING WORKFLOW

FROM LAUNCH TO REPLAY

Record raw. Rebuild smart.

RoCam stores the raw camera feed and per-frame telemetry separately, then reconstructs stabilized playback and downloads on demand.

TRIGGER
Browser UI
Start / stop from Mission Control

STORAGE
AVI + JSONL
Video frames plus OSD telemetry

OUTPUT
Preview / Download
Rendered after the flight event

WHY THIS SPLIT MATTERS

- Capture stays lightweight during launch instead of doing heavy replay rendering in the critical path.
- The saved telemetry lets us re-apply OSD, timestamps, zoom transforms, and stabilization later for post-flight review.

1. CAPTURE

Raw video branch to video.avi

The CV pipeline swaps in a recording sink and writes the camera stream without waiting for replay rendering.

2. LOG

Per-frame OSD data to log.txt

Each frame stores pan, tilt, focal length, FPS, timestamps, and stabilization transforms as JSONL telemetry.

3. REPLAY

Separate transcode for stabilized review

Preview and download are reconstructed later with the same OSD and shader path, producing a polished replay stream for analysis.

481

AUTOMATED TESTS PASSED

Backend (pytest)

313 Unit Tests

88% Coverage

Frontend (Vitest)

168 Component Tests

86% Coverage

CI Pipeline

GitHub Actions on every PR

100% Pass Rate

PROJECT CHALLENGES

Real-time Latency
Python's GIL blocking CV loops.

→

Solution
Shared-memory IPC bypassing GIL entirely.

Micro-Object Detection
Rockets vanishing into background noise.

→

Solution
YOLO26s-P2 Head & Hard-negative mining.

Hardware Sync
UART dropping packets at 115200 baud.

→

Solution
Logic analyzer debug + custom CRC-8 checksums.

Designed to scale from 200m hobby rockets to 3km+ high-power launches.

Performance

- 4K @ 60fps tracking
- Authentic ML Architecture
- Optical Auto-focus

NEXT STEP

Hardware Scaling

- Full-size Gimbal Mounts
- Multi-camera Arrays
- Weatherproof Enclosures

Software UX

- Mobile-native Apps
- Post-flight Analytics
- AR Telemetry Overlay

Built with operator feedback, not just lab assumptions.

MCMASTER ROCKETRY TEAM

"A more direct and intuitive way to move the gimbal."

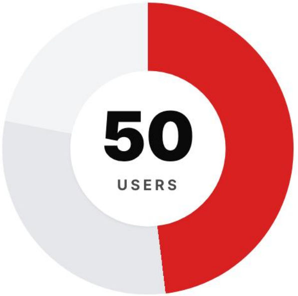
Captured in stakeholder walkthrough. Turned into drag-to-move control on the live preview.

Direct Control
Drag-to-move framing added beside the D-pad.

Clear Status
More visible armed, target-lost, storage, and network states.

Stable Recording
Pause/resume behavior refined for download and recording.

MCMASTER ROCKETRY TEAM · 50 MEMBERS



Autonomous Tracking
24 members · 48%

Manual Override
15 members · 30%

Video Recording
11 members · 22%

Thank You.

Questions?

https://github.com/SpaceY-Labs/RoCam

WITH THANKS TO

SUPERVISOR

Dr. Shahin Sirouspour

Faculty guidance and project support

INSTRUCTOR

Dr. Spencer Smith

Thanks to the course for the structure and direction.

TEACHING ASSISTANT

Tanya Djavaherpour

Thanks for the feedback throughout the project.

TEAM #3 SPACEY | MCMASTER UNIVERSITY SFWRENG 4G06